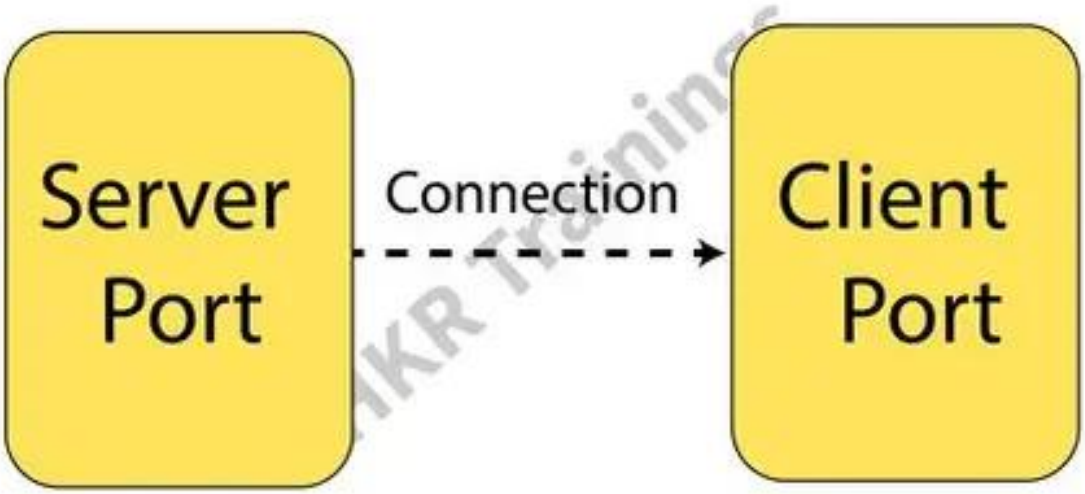
REPORTE DE PRÁCTICA 03: Manejo de errores y excepciones en programación de sockets

Experiencia Educativa: Programación en Red

Alumno: Jorge Gael Hernández Marcial

Fecha: 08/10/2026



Objetivo de la Práctica

Diseñar e implementar mecanismos de manejo de errores y excepciones en aplicaciones de red (modelo Cliente-Servidor) utilizando el lenguaje Python. El propósito es construir sistemas robustos y tolerantes a fallos que no se detengan ante caídas de la red o desconexiones abruptas de los clientes, aplicando correctamente las estructuras try/except y protegiendo los recursos compartidos en entornos concurrentes mediante el uso de hilos y candados de sincronización.

Marco Teórico / Fundamentos de Red

- Errores vs. Excepciones: Un error de sintaxis impide que el programa inicie debido a instrucciones mal escritas. Una excepción es un error en tiempo de ejecución (como conectarse a un servidor apagado) que ocurre a pesar de tener un código sintácticamente correcto.
- Estructura de Manejo (try / except / else / finally):
    - try: Encierra el código susceptible a fallos.
    - except: Captura y maneja una excepción específica. Se deben ordenar de lo más específico a lo más general.
    - else: Se ejecuta únicamente si el bloque try concluyó sin errores.
    - finally: Se ejecuta invariablemente para liberar recursos (como cerrar sockets o archivos).
- Jerarquía de Excepciones de Red (OSError):
    - ConnectionRefusedError: Ocurre al intentar conectarse a un puerto donde no hay ningún servicio escuchando.
    - TimeoutError: Se lanza cuando funciones como recv() o recvfrom() agotan el tiempo límite de espera (settimeout) sin recibir datos.
    - ConnectionResetError: Sucede cuando el extremo remoto de la conexión cierra el socket de manera abrupta.
    - socket.gaierror: Se presenta cuando el sistema no puede resolver una dirección IP o nombre de host.

Desarrollo y Metodología

A. Gestión de Hilos y Estado Local (Ejercicios 1 y 2) Se desarrollaron scripts para dominar la concurrencia. Se crearon hilos a partir de una lista de tuplas, utilizando .join() para asegurar que el programa principal espere la finalización de todos los procesos secundarios. Asimismo, se implementó un servidor de eco donde cada hilo

mantiene una variable local independiente (contador de mensajes), garantizando que clientes simultáneos no interfieran entre sí.

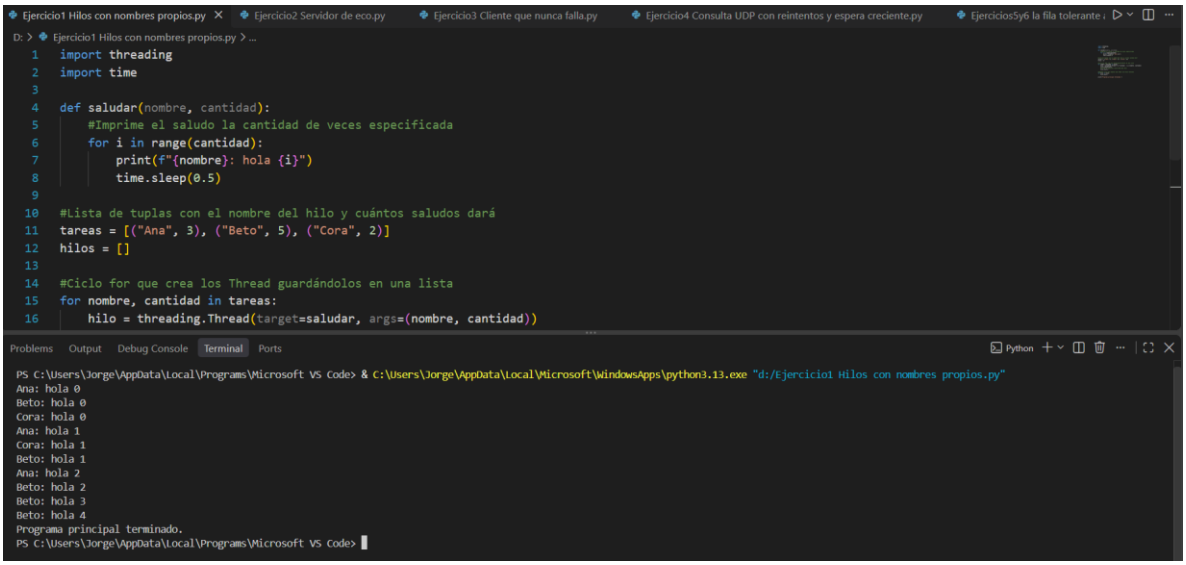
B. Cliente TCP Robusto (Ejercicio 3) Se programó un cliente TCP con manejo exhaustivo de excepciones. Mediante bloques try/except específicos, el cliente es capaz de identificar y reportar problemas exactos: IPs inválidas (gaierror), servidores inalcanzables (ConnectionRefusedError) o desconexiones (ConnectionResetError). En lugar de colapsar y mostrar el traceback al usuario, el programa emite un mensaje claro y aborta la ejecución limpiamente con sys.exit(1).

C. Tolerancia a Fallos en UDP (Ejercicio 4) Debido a que UDP no garantiza la entrega de paquetes, se integró un mecanismo de reintentos acotado usando un bucle for. Al capturar la excepción TimeoutError, el cliente espera un tiempo que crece exponencialmente en cada intento (2 ** (intento - 1)) antes de reenviar el datagrama, evitando así ciclos infinitos si el servidor está definitivamente inactivo.

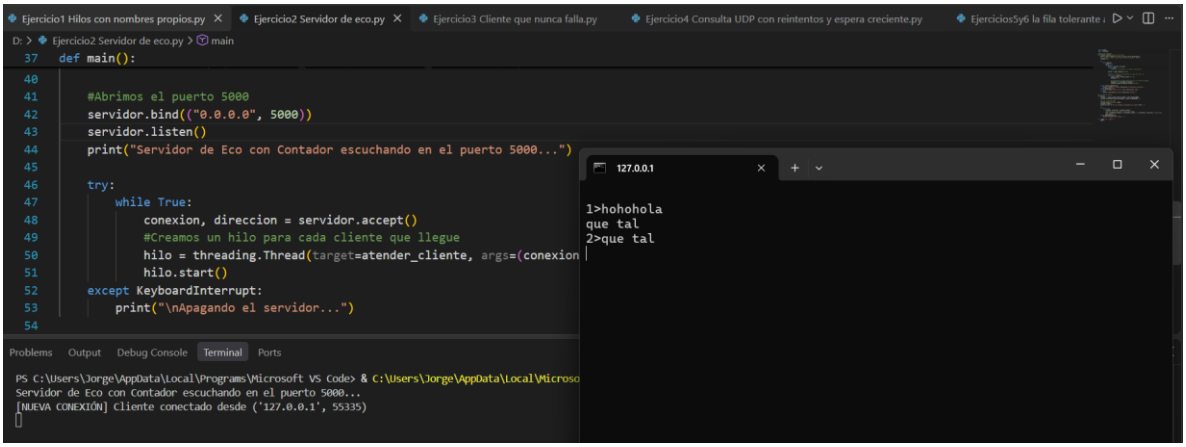
D. Servidor Tolerante a Fallos y Registro Log (Ejercicios 5 y 6) Se perfeccionó el sistema de asignación de turnos ("LA FILA"). La función de atención delegada a cada hilo (tomar_turno) fue encapsulada en un try/except. Si un cliente envía datos que violan el protocolo (no comienzan con TURNO>) o manda bytes que no son UTF-8 válidos, la excepción es capturada sin afectar el ciclo principal de aceptación de otros clientes. Finalmente, se programó la función registrar_error, que anexa estos incidentes a un archivo de texto (errores.log) con su marca de tiempo. Para evitar escrituras cruzadas o archivos corruptos, esta operación se protegió con un candado de exclusión mutua (candado_log).

Evidencias de Ejecución (Capturas de Pantalla)

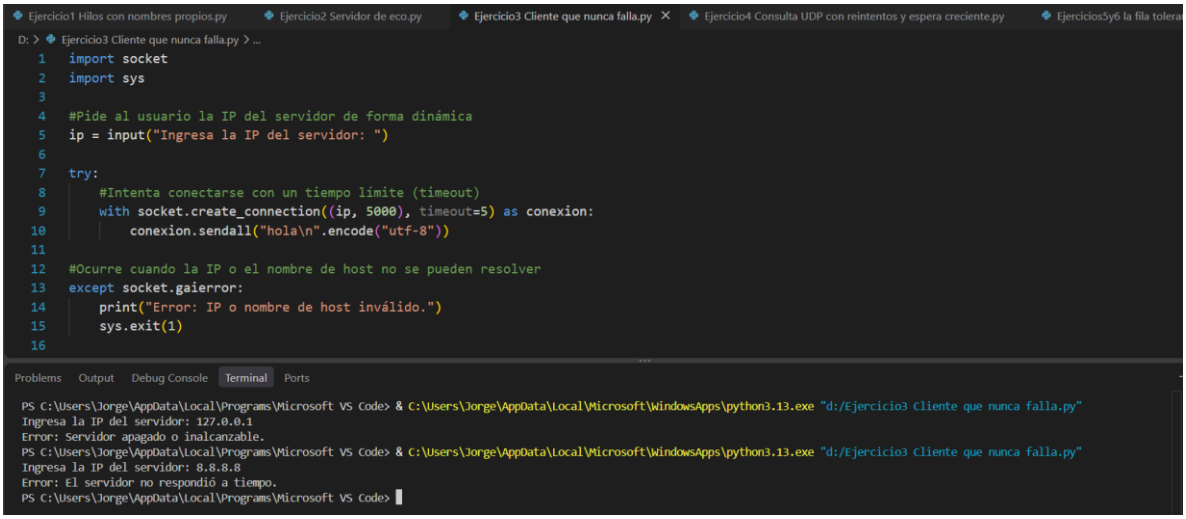
Ejercicio 1.



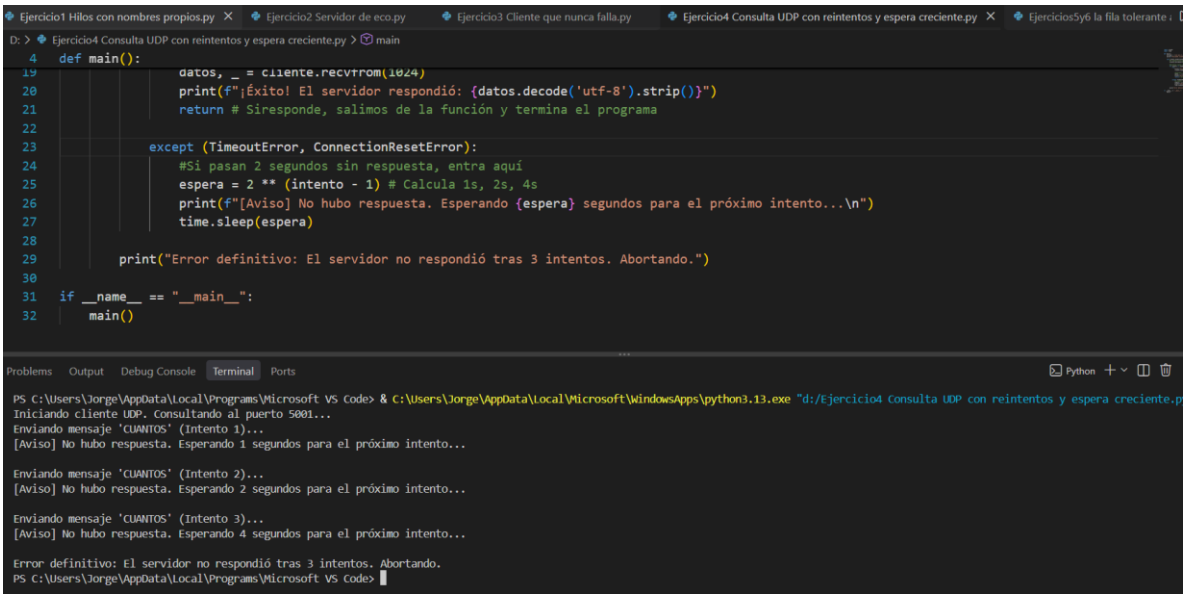
Ejercicio 2.



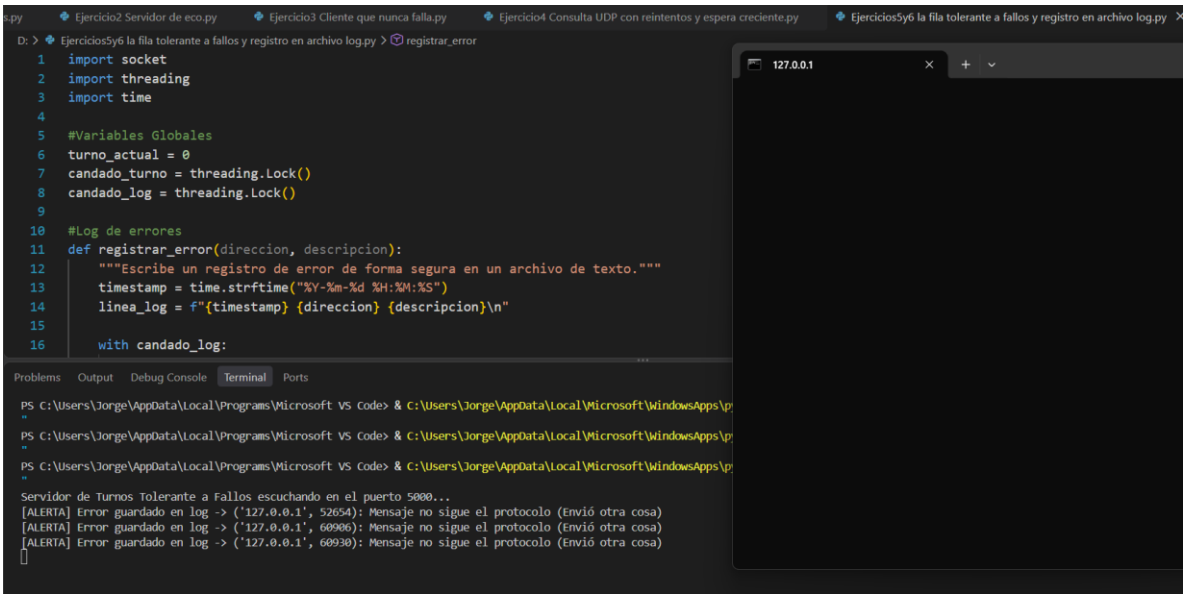
Ejercicio 3.



Ejercicio 4.



Ejercicio 5 y 6.



Conclusiones

El desarrollo y depuración de esta práctica demostró que la programación en red conlleva desafíos técnicos prácticos que van mucho más allá de la teoría. Durante la implementación se experimentaron y solucionaron múltiples fallos que resultaron ser aprendizajes fundamentales sobre el comportamiento del sistema operativo y la concurrencia:

- Ciclo de vida de los hilos: Se comprobó empíricamente que instanciar hilos secundarios (Daemon Threads) no es suficiente si el hilo principal no gestiona su permanencia (mediante funciones bloqueantes o el uso de

.join()). La omisión de estas llamadas provocaba el cierre prematuro e inmediato de los servidores sin llegar a procesar peticiones.

- Gestión de estados de red: El error ConnectionRefusedError (WinError 10061) evidenció la necesidad de una sincronización estricta entre los extremos, demostrando qué ocurre a nivel de sistema operativo cuando un cliente TCP intenta enlazar con un puerto donde no hay un proceso en estado de escucha activa (listen).
- Peculiaridades del Sistema Operativo en UDP: Un descubrimiento crítico fue la reacción de Windows ante datagramas UDP huérfanos en la interfaz local (127.0.0.1). A diferencia del estándar teórico donde UDP simplemente pierde el paquete, el sistema operativo forzaba una interrupción (ConnectionResetError / WinError 10054) al intentar leer con recvfrom(). Esto obligó a adaptar el manejo de excepciones (except (TimeoutError, ConnectionResetError):) para asegurar que el algoritmo de reintentos crecientes funcionara correctamente bajo este entorno.